

PDEForge: A Unified Framework for Generating PDE Datasets for Operator Learning

Peter Yatsyshin

January 2026

Summary

Operator learning has emerged as a transformative approach for accelerating numerical simulations of physical systems governed by partial differential equations (PDEs). Neural operators learn mappings between infinite-dimensional function spaces, enabling fast surrogate models that can replace computationally expensive traditional solvers. However, generating suitable training datasets remains a practical bottleneck: existing benchmark datasets are often fixed in resolution and parameters, while generating custom data requires implementing solvers from scratch for each PDE type.

PDEForge addresses this gap by providing a unified Python framework for generating PDE datasets across diverse physical systems. The package implements a consistent API for 15+ PDE models spanning 1D and 2D domains, from scalar diffusion equations to turbulent flows around obstacles (Figure 1). PDEForge is designed with uncertainty quantification (UQ) as a primary concern, including built-in support for calibration splits required by methods such as conformal prediction.

Statement of Need

The field of operator learning has seen rapid growth, with architectures such as Fourier Neural Operators (Li et al. 2020), DeepONet (Lu et al. 2021), and neural operator transformers (Kovachki et al. 2023) demonstrating strong performance on PDE surrogate modeling tasks. A persistent challenge is the fragmented landscape of training data:

1. **Static benchmarks:** Existing datasets like those in PDEBench (Takamoto et al. 2022) provide fixed-resolution data with predetermined parameters, limiting researchers who need different configurations for systematic studies.
2. **Inconsistent interfaces:** Each PDE implementation typically has its own API and data format, making systematic comparisons across problems tedious and error-prone.
3. **UQ considerations:** Methods for uncertainty quantification in neural operators (Zou et al. 2023) require dedicated calibration sets, which existing benchmarks do not typically provide.
4. **Complex geometries:** Flow problems around obstacles require finite element methods and mesh generation, creating significant implementation overhead for researchers focused on machine learning.

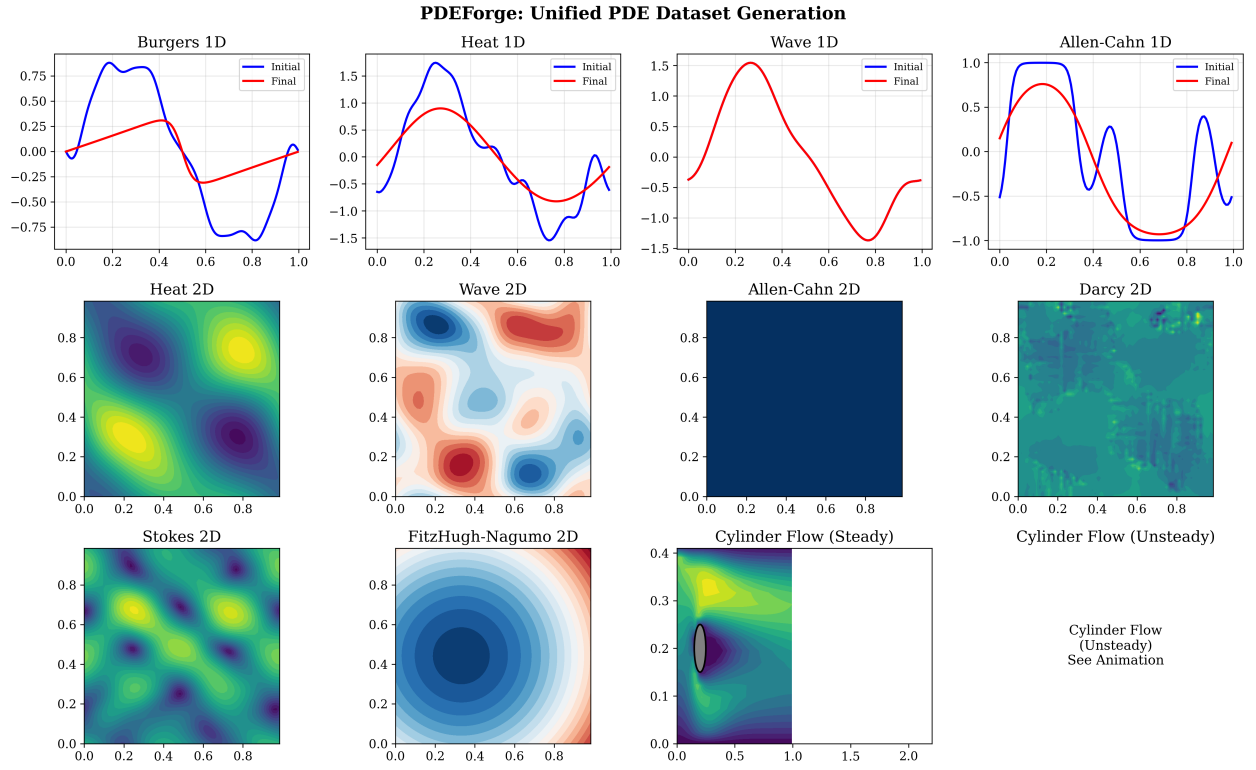


Figure 1: Overview of PDE models available in PDEForge. Top row: 1D time-dependent equations (Burgers, Heat, Wave, Allen-Cahn). Middle row: 2D scalar field problems (Heat, Wave, Allen-Cahn, Darcy). Bottom row: 2D vector field and flow problems (Stokes, FitzHugh-Nagumo, steady and unsteady cylinder flow).

PDEForge provides a unified solution: a single `generate_dataset()` function works across all supported PDE models, returning data in a consistent format with metadata and optional train/validation/calibration/test splits.

Implemented Models

PDEForge includes a comprehensive set of PDE models organized by dimensionality and physical domain:

1D Time-Dependent PDEs:

- Burgers equation (nonlinear advection-diffusion with shock formation)
- Heat equation (linear diffusion)
- Wave equation (hyperbolic wave propagation)
- Allen-Cahn equation (phase field dynamics)
- FitzHugh-Nagumo system (excitable media)
- Stochastic heat equation (diffusion with multiplicative noise)

2D Steady and Time-Dependent PDEs:

- Heat, Wave, and Allen-Cahn equations (extensions of 1D models)
- Darcy flow (porous media with heterogeneous permeability)
- Stokes flow (incompressible viscous flow)
- FitzHugh-Nagumo 2D (spiral wave formation)

FEniCSx-Based Flow Models:

- Steady cylinder flow (Navier-Stokes around obstacle)
- Unsteady cylinder flow (vortex shedding dynamics)
- Parameterized cylinder flow (variable obstacle position)
- Turbulent cylinder flow (Smagorinsky LES for high Reynolds numbers)

Figure 2 demonstrates the unsteady cylinder flow model capturing the von Kármán vortex street, a canonical benchmark for fluid dynamics and neural operator evaluation.

Key Features

Unified API

All PDEForge models share an identical interface:

```
from pdeforge import generate_dataset

dataset = generate_dataset(
    model="burgers_1d",
    n_samples=1000,
    resolution={"x": 256},
    params={"viscosity": 0.01},
    seed=42,
)
```

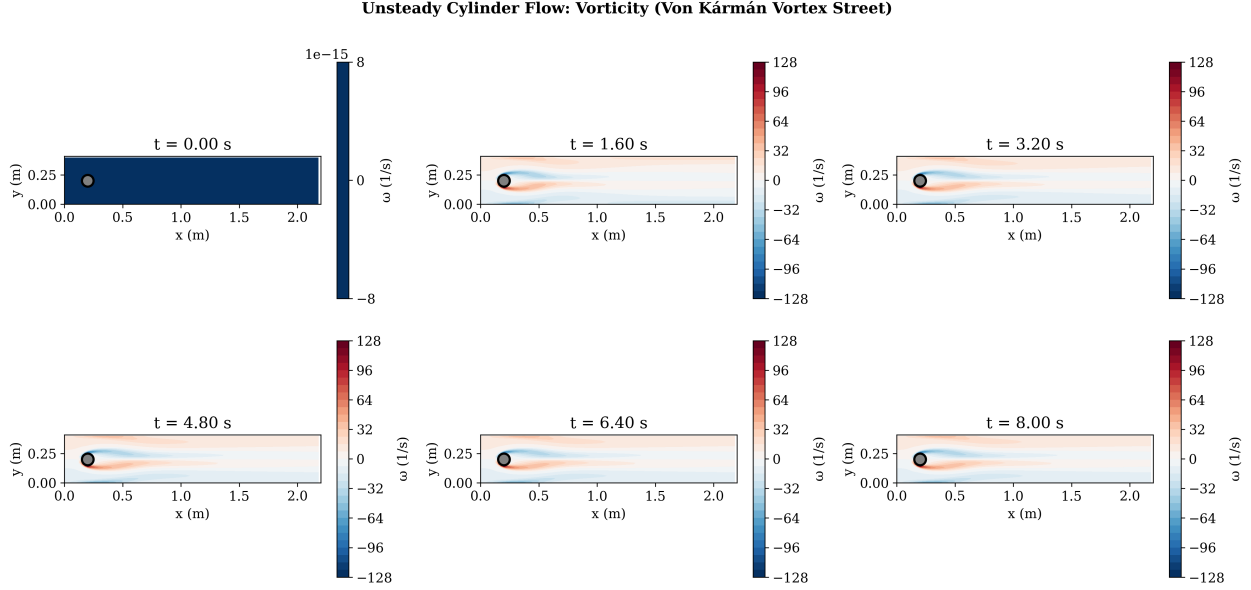


Figure 2: Time evolution of vorticity in the unsteady cylinder flow model, showing development of the von Kármán vortex street. The simulation captures vortex shedding dynamics at moderate Reynolds number using FEniCSx finite elements.

UQ-Ready Data Splits

PDEForge natively supports four-way splits for uncertainty quantification workflows:

```
splits = dataset.split(train=0.6, val=0.15, cal=0.15, test=0.1)
```

The calibration split enables conformal prediction and other post-hoc UQ methods.

Extensibility

New models integrate through a registry pattern, allowing community contributions without modifying core code:

```
from pdeforge.core.registry import register_model

@register_model("my_custom_pde")
class MyPDE(PDEModel):
    ...
```

Implementation

PDEForge is implemented in Python with dependencies on NumPy, SciPy, and Matplotlib. Spectral models use FFT-based pseudo-spectral methods for efficiency. Complex-geometry flow models use the FEniCSx finite element library (Baratta et al. 2023) with gmsh for mesh generation.

The architecture cleanly separates PDE-specific logic from common infrastructure (dataset management, I/O, visualization), facilitating maintenance and community contributions.

Availability

PDEForge is available on GitHub under the MIT license at github.com/imperial-qore/PDEForge. Documentation, installation instructions, and Jupyter notebook tutorials are provided.

Acknowledgements

PY acknowledges support from The Alan Turing Institute Turing Research Fellowship.

References

- Baratta, Igor A, Joseph P Dean, Jørgen S Dokken, et al. 2023. “DOLFINx: The Next Generation FEniCS Problem Solving Environment.” *Zenodo*, ahead of print. <https://doi.org/10.5281/zenodo.10447666>.
- Kovachki, Nikola, Zongyi Li, Burigede Liu, et al. 2023. “Neural Operator: Learning Maps Between Function Spaces with Applications to PDEs.” *Journal of Machine Learning Research* 24 (89): 1–97.
- Li, Zongyi, Nikola Kovachki, Kamyar Azizzadenesheli, et al. 2020. “Fourier Neural Operator for Parametric Partial Differential Equations.” *arXiv Preprint arXiv:2010.08895*.
- Lu, Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. 2021. “Learning Nonlinear Operators via DeepONet Based on the Universal Approximation Theorem of Operators.” *Nature Machine Intelligence* 3 (3): 218–29.
- Takamoto, Makoto, Timothy Praditia, Raphael Leber, et al. 2022. “PDEBench: An Extensive Benchmark for Scientific Machine Learning.” *Advances in Neural Information Processing Systems* 35: 1596–611.
- Zou, Yibo, Nikola Kovachki, Kamyar Azizzadenesheli, and Andrew Stuart. 2023. “Uncertainty Quantification for Neural Operator Learning.” *arXiv Preprint arXiv:2302.02319*.